Particelle Cariche in un Campo Elettromagnetico

Gabriele Cembalo

8 gennaio 2026

Abstract

In questa esperienza ci si è occupati dello studio dell'effetto Hall. Prima di effettuare qualsiasi misura, è stato verificato che il campo magnetico generato nel traferro del magnete utilizzato fosse qualitativamente omogeneo entro una regione della sezione del magnete. Successivamente, è stata variata la corrente circolante nelle bobine per caratterizzare due cicli di isteresi completi. Dalla forma di tale ciclo è stato dedotto che il magnete fosse dolce, mentre dalle rispettive salite e discese di queste curve sono stati estrapolati i parametri della retta $B(i)$, divenuta calibrazione del set-up. In seguito, ci si è occupati del calcolo dei coefficienti di Hall di un campione di Germanio, ottenuti prima a campo magnetico costante e poi a corrente costante ed essi sono risultati compatibili. Per farlo è stato considerato anche un fondo di tensione, di natura ohmica, dovuta al disallineamento dei contatti nel semiconduttore. Per concludere, conoscendo ora il coefficiente di Hall del Germanio, è stata ricavata la mobilità dei portatori avendo prima misurato la resistenza del campione.

Indice

1 Introduzione **2**

2 Teoria **2**

3 Metodi numerici utilizzati **3**
- 3.1 Metodo Runge-Kutta del quart'ordine . . . 3
- 3.2 Metodo Boris . . . 3

4 Risultati nelle diverse configurazioni **4**
- 4.1 Implementazione codice . . . 4
- 4.2 Rotazione . . . 5
- 4.3 Campo elettrico e magnetico paralleli . . . 7
- 4.4 Campo elettrico e magnetico perpendicolari . . . 8
- 4.5 Configurazione ad X . . . 8

5 Conclusioni **8**

A Inserting Code **9**

Bibliografia **16**

---

1 Introduzione

In questo progetto si studia il moto di una o più particelle cariche, non relativistiche, immerse in un campo elettromagnetico costante. Vengono analizzate diverse configurazioni di campo. Nella sezione 2 analizzeremo la teoria alla base del problema. Nella sezione 3 vengono spiegate meglio le motivazioni per cui un approccio numerico è necessario/preferibile rispetto uno analitico; inoltre vengono analizzati i due metodi di risoluzione dell'equazione differenziale proposti, ovvero il metodo di Runge-Kutta del quarto ordine e l'algoritmo di Boris del secondo ordine. Studiamo le diverse configurazioni di campo elettromagnetico e i diversi risultati nella sezione 4. In particolare, considerando una particella singola, trattiamo le configurazioni: campo magnetico costante e campo elettrico nullo; campo elettrico e magnetico costanti e paralleli; campo elettrico e magnetico perpendicolari. Nell'ultimo caso proposto vengono studiate più particelle in una configurazione ad X point. Per concludere, nella sezione 5 traiamo le conclusioni. È presente nell'appendice A il codice utilizzato.

2 Teoria

Come già accennato nell'introduzione, lo scopo di questo progetto è quello di studiare il modo di una o più particelle, non relativistiche, in un campo elettromagnetico fissato. Vogliamo risolvere l'equazione di Lorentz:

$$\frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \frac{e}{mc}\left(c\mathbf{E} + \mathbf{v}\times\mathbf{B}\right) \tag{1}$$

scritta nel sistema c.g.s., in cui e è la carica della particella, m e $\mathbf{v}$ rispettivamente la sua massa e velocità, c la velocità della luce nel vuoto e $\mathbf{E}$ e $\mathbf{B}$ i campi elettrico e magnetico. Se consideriamo $e = -4.80\cdot 10^{-10}$ statC la carica elementare di un elettrone e la massa $m = m_e = 9.11\cdot 10^{-28}$ g la massa dell'elettrone, allora ricordando anche il valore $c = 3\cdot 10^{10}$ cm/s otteniamo il fattore:

$$\alpha = \frac{e}{m_e c} = 1.76\cdot 10^{7}\ \mathrm{cm}^{1/2}\,\mathrm{g}^{-1/2}\,\mathrm{s}^{-1} \tag{2}$$

davanti alla forza di Lorentz (1). Per comodità si preferisce lavorare con quantità adimensionali (linguaggio macchina). Se poniamo $\alpha = 1$, allora dobbiamo considerare una scala temporale in 10^{-7} s per avere le stime delle nostre quantità fisiche dai risultati della simulazione. Infatti, se scriviamo le quantità fisiche, ad esempio una generica b, come $b = b_{cgs}b_c$, in cui b_{cgs} è una costante in unità c.g.s. che rappresenta l'unità di scala sceltà, mentre b_c è la quantità adimensionale da usare nel codice, allora otteniamo l'equazione del moto:

$$\frac{l_{cgs}}{t_{cgs}^2}\frac{\mathrm{d}\mathbf{v}_c}{\mathrm{d}t_c} = \frac{e}{m_e c}\left[\frac{m_e c}{e}\frac{l_{cgs}}{t_{cgs}^2}\mathbf{E}_c + \left(\frac{l_{cgs}}{t_{cgs}}\mathbf{v}_c\right)\times\left(\frac{m_e}{e}\frac{1}{t_{cgs}}\mathbf{B}_c\right)\right] \tag{3}$$

mentre l'equazione adimensionale che dobbiamo risolvere nel codice è:

$$\frac{\mathrm{d}\mathbf{v}_c}{\mathrm{d}t_c} = \left(\mathbf{E}_c + \mathbf{v}_c\times\mathbf{B}_c\right). \tag{4}$$

Sappiamo bene che l'equazione (4) non descrive alcun sistema fisico, ma contiene delle quantità collegate alle osservabili del nostro problema tramite lo scaling dalla costante α. Per ottenere le informazioni fisiche dovremo prendere i risultati delle nostre simulazioni e applicare la trasformazione $b = b_{cgs}b_c$ con $b_{cgs} = 1/\alpha$. Ad esempio, per il tempo avremo:

$$t = t_{cgs}t_c = \frac{m_e c}{e}\,t_c\ \mathrm{s} = \frac{9.11\cdot 10^{-28}\cdot 3\cdot 10^{10}}{4.80\cdot 10^{-10}}\,t_c\ \mathrm{s} \approx 5.69\cdot 10^{-8}\,t_c\ \mathrm{s}. \tag{5}$$

Può risultare utile per l'implementazione del codice scomporre l'equazione (4) nelle tre componenti spaziali:

$$\begin{aligned}
\frac{\mathrm{d}v_{c,x}}{\mathrm{d}t} &= \left(E_{c,x} + v_{c,y}B_{c,z} - v_{c,z}B_{c,y}\right)\\
\frac{\mathrm{d}v_{c,y}}{\mathrm{d}t} &= \left(E_{c,y} + v_{c,z}B_{c,x} - v_{c,x}B_{c,z}\right)\\
\frac{\mathrm{d}v_{c,z}}{\mathrm{d}t} &= \left(E_{c,z} + v_{c,x}B_{c,y} - v_{c,y}B_{c,x}\right)
\end{aligned} \tag{6}$$

in cui notiamo che il campo magnetico contribuisce solo per la variazione della direzione della velocità, e non del modulo di essa, per via del prodotto vettoriale.

3 Metodi numerici utilizzati

La necessità di adottare un approccio numerico per questo problema deriva principalmente dalla natura intrinseca delle equazioni del moto in presenza di campi elettromagnetici non banali. Sebbene per configurazioni estremamente semplificate sia possibile determinare una soluzione analitica, l'integrazione delle equazioni di Lorentz diventa rapidamente complessa non appena si considerano campi spazialmente variabili o configurazioni multi-particella. In questi casi, infatti, il sistema di equazioni differenziali ordinarie risulta accoppiato e spesso non risolvibile in forma chiusa tramite funzioni elementari.

L'utilizzo di algoritmi numerici permette quindi di superare i limiti della trattazione analitica, consentendo lo studio di traiettorie in geometrie arbitrarie, come nel caso del punto a X, e garantendo al contempo il controllo dell'accuratezza e della conservazione delle grandezze fisiche fondamentali su tempi di integrazione prolungati.

In questo progetto, i metodi numerici utilizzati sono il metodo di Runge-Kutta del quarto ordine e l'algoritmo di Boris. Le principali fonti di riferimento per questi metodi sono le slide delle lezioni; per quanto riguarda il metodo di Boris, si è fatto riferimento anche alla trattazione di Qin et al. [2], consultabile al seguente link. Una spiegazione più intuitiva, didattica e operativa è inoltre disponibile in [1].

3.1 Metodo Runge-Kutta del quart'ordine

Il **metodo di Runge-Kutta del quart'ordine** viene utilizzato per la risoluzione numerica di problemi ai valori iniziali del tipo:

$$\frac{\mathrm{d}Y}{\mathrm{d}t} = R(t, Y) \qquad , \qquad Y(t_0) = Y_0. \tag{7}$$

Tale metodo è un metodo iterativo, a singolo passo, che utilizza solo l'informazione sull'intervallo corrente e non sui precedenti. Tale metodo si basa sull'espansione di Taylor, sull'approssimazione dell'integrale con la regola di Simpson ed è un buon algoritmo in termini di accuratezza e stabilità; infatti ha un'accuratezza locale dell'ordine $\mathcal{O}(\Delta t^5)$, che si traduce in un errore globale dell'ordine $\mathcal{O}(\Delta t^4)$. Il metodo richiede quattro valutazioni della *Right-Hand-Side function* per ogni passo temporale:

$$\begin{aligned}
k_1 &= R\left(t_n, Y_n\right) \\
k_2 &= R\left(t_n + \frac{\Delta t}{2}, Y_n + \frac{\Delta t}{2} k_1\right) \\
k_3 &= R\left(t_n + \frac{\Delta t}{2}, Y_n + \frac{\Delta t}{2} k_2\right) \\
k_4 &= R\left(t_n + \Delta t, Y_n + \Delta t\, k_3\right) \\
Y_{n+1} &= Y_n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).
\end{aligned} \tag{8}$$

Le quattro valutazioni della funzione R vengono effettuate in punti diversi dell'intervallo di discretizzazione temporale. In particolare, k_1 e k_4 corrispondono rispettivamente agli estremi iniziale e finale dell'intervallo, mentre k_2 e k_3 sono calcolati nel punto medio, fornendo una stima più accurata dell'andamento della soluzione.

La combinazione pesata dei coefficienti k_i deriva dall'approssimazione dell'integrale mediante la regola di Simpson e consente di ottenere un'elevata accuratezza senza aumentare il numero di passi temporali.

3.2 Metodo Boris

Il secondo metodo utilizzato è il **metodo di Boris**, che è un integratore simplettico e un'estensione dell'algoritmo Position-Verlet. Questo metodo viene utilizzato per la risoluzione numerica del problema ai

valori iniziali associato alle equazioni del moto di una particella carica soggetta a campi elettromagnetici, ovvero a problemi in cui la forza dipende dalla velocità, come nel caso descritto dal sistema:

$$\begin{cases} \dfrac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{v} \\ \dfrac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \dfrac{q}{m}\Big(\mathbf{E} + \mathbf{v} \times \mathbf{B}\Big) \end{cases} \quad , \qquad \mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{v}(t_0) = \mathbf{v}_0 \tag{9}$$

in cui $\mathbf{E}$ e $\mathbf{B}$ indicano rispettivamente il campo elettrico e magnetico valutati nella posizione della particella. Il metodo di Boris è un metodo iterativo, esplicito e a singolo passo, che utilizza esclusivamente le informazioni relative all'intervallo temporale corrente e non sui passi precedenti. A differenza dei metodi di tipo Runge-Kutta, esso non si basa su un'espansione di Taylor né su una quadratura diretta dell'integrale, ma su una scomposizione simmetrica dell'operatore di avanzamento temporale, che separa l'azione del campo elettrico da quella del campo magnetico.

Il metodo è di secondo ordine in accuratezza globale, con errore locale dell'ordine $\mathcal{O}(\Delta t^3)$, ed è particolarmente apprezzato per le sue proprietà di stabilità e per la capacità di preservare le invarianti fisiche del sistema. In particolare, in assenza di campo elettrico, il metodo conserva l'energia cinetica della particella, riproducendo correttamente il moto di rotazione indotto dal campo magnetico.

L'aggiornamento della velocità viene effettuato attraverso una sequenza di tre sotto-passi, mentre la posizione viene aggiornata come:

$$\begin{aligned} \mathbf{x}_{n+\frac{1}{2}} &= \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{v}_n \\ \mathbf{v}_- &= \mathbf{v}_n + \frac{\Delta t}{2}\mathbf{E}_{n+\frac{1}{2}} \\ \mathbf{v}_+ &= \mathbf{v}_- + 2\frac{\mathbf{v}_- + \mathbf{v}_- \times \frac{\Delta t}{2}\mathbf{B}_n}{1 + \frac{\Delta t^2}{4}\mathbf{B}_n^2} \times \left(\frac{\Delta t}{2}\mathbf{B}_n\right) \\ \mathbf{v}_{n+1} &= \mathbf{v}_+ + \frac{\Delta t}{2}\mathbf{E}_{n+\frac{1}{2}} \\ \mathbf{x}_{n+1} &= \mathbf{x}_{n+\frac{1}{2}} + \frac{\Delta t}{2}\mathbf{v}_{n+1}. \end{aligned} \tag{10}$$

Gli aggiornamenti intermedi della velocità vengono effettuati in punti diversi dell'intervallo di discretizzazione temporale. In particolare, i termini $\mathbf{v}_-$ e $\mathbf{v}_{n+1}$ rappresentano gli effetti del campo elettrico applicati rispettivamente all'inizio e alla fine del passo temporale, mentre l'aggiornamento centrale $\mathbf{v}_+$ descrive l'azione del campo magnetico, trattata come una rotazione nel punto medio dell'intervallo.

La struttura simmetrica del metodo consente una buona conservazione delle invarianti fisiche del sistema, in particolare dell'energia cinetica in assenza di campo elettrico, garantendo stabilità numerica anche per integrazioni di lungo termine.

N.B. Nell'implementazione dell'algoritmo nell'Appendice A viene utilizzata la notazione (e le quantità) di [1]. Ci serviremo delle quantità:

$$t = \frac{\Delta t}{2}\mathbf{B} \quad , \quad \mathbf{v}' = \mathbf{v}_- + \mathbf{v}_- \times \mathbf{t} \quad , \quad \mathbf{s} = \frac{2\mathbf{t}}{1 + t^2} \tag{11}$$

così viene comodo calcolare:

$$\mathbf{v}_+ = \mathbf{v}_- + \mathbf{v}' \times \mathbf{s}. \tag{12}$$

4 Risultati nelle diverse configurazioni

Analizziamo in questa sezione i risultati ottenuti nelle varie configurazioni dei campi e condizioni iniziali. Viene inserita una prima sezione in cui viene spiegata (brevemente) la struttura del codice implementato nell'Appendice A.

4.1 Implementazione codice

Sono stati definiti i parametri:

- `T_L` indica il tempo totale di integrazione.
- `N_Part` indica il numero di particelle.
- `X_L` ed `Y_L` per indicare il dominio quatrato del caso a multi-particelle.

Per i casi a singola particella - dunque le configurazioni di semplice rotazione, e di campi elettrico e magnetico parallelo e perpendicolari - è stato definito il vettore `t_vec[]` per contenere diversi passi temporali con cui fare le simulazioni. Si sono utilizzati gli intervalli:

$$\Delta t = \big(1.0\,,\,0.1\,,\,0.01\big).$$

Nota. All'interno del codice è presenta anche un altro vettore `t_vec[]` che però contiene gli intervalli temporali utilizzati per verificare la non conservazione delle proprietà geometriche del sistema da parte del metodo di Runge-Kutta. È dunque necessario fare attenzione a commentare il vettore di spaziatura temporale che non si vuole utilizzare.

Le funzioni implementate, oltre ai metodi di risoluzione di Runge-Kutta del quart'ordine e di Boris, sono:

- La funzione `EB_Fields`, in cui a seconda della configurazione scelta viene riempito un vettore a 6 componenti con il campo elettrico e mangetico. In particolare si è scelto di mettere nelle prime 3 componenti il campo elettrico E_x, E_y, E_z, mentre nelle restanti il campo magnetico B_x, B_y e B_z.
- La funzione `InitialCondition`, in cui per ogni situazione analizzata vengono settate le condizioni iniziali al vettore delle soluzioni $Y[i]$, anch'esso a 6 componenti che sono: le prime 3 le posizioni x, y e z, mentre le restanti sono le componenti della velocità v_x, v_y e v_z.
- La funzione `RHSFunc` che è la funzione del Right-Hand-Side function dell'equazione differenziale che stiamo studiando. Ovviamente differenzia le diverse configurazioni.

4.2 Rotazione

Come prima configurazione si è utilizzata la situazione in cui il vettore di campo elettrico e magentico sono rispettivamente:

$$\mathbf{E} = \big(0.0\,,\,0.0\,,\,0.0\big) \qquad ; \qquad \mathbf{B} = \big(0.0\,,\,0.0\,,\,1.0\big) \tag{13}$$

e per la quale l'equazione del moto in componenti (6) diventa:

$$\begin{aligned} \frac{\mathrm{d}v_x}{\mathrm{d}t} &= v_y B_z \\ \frac{\mathrm{d}v_y}{\mathrm{d}t} &= -v_x B_z \\ \frac{\mathrm{d}v_z}{\mathrm{d}t} &= 0. \end{aligned} \tag{14}$$

Come condizioni iniziali su posizione e velocità delle particelle si è scelto:

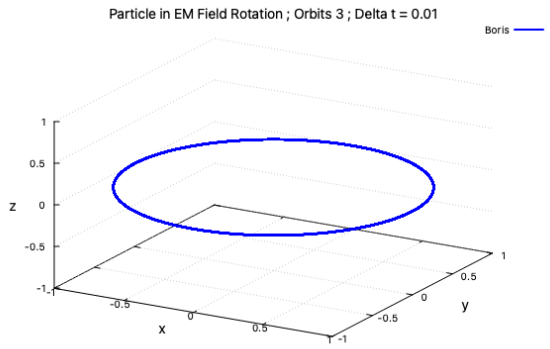
$$\mathbf{x} = \big(0.0\,,\,1.0\,,\,0.0\big) \qquad ; \qquad \mathbf{v} = \big(1.0\,,\,0.0\,,\,0.0\big). \tag{15}$$

La soluzione delle equazioni (14) è la curva:

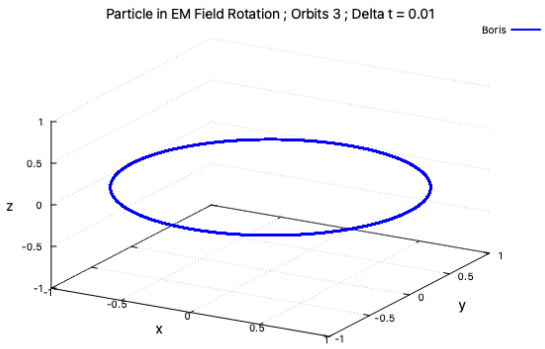
$$(x, y) = (\sin(\theta)\,,\,\cos(\theta)) \tag{16}$$

in cui θ è un parametro, che possiamo interpretare come tempo di simulazione $\theta = t$. La nostra particella, secondo la soluzione dell'equazione del moto segue una traiettoria circolare (in senso orario) nel piano xy. Nella figura 1 si è verificata la validità dei due metodi utilizzati, confrontando la soluzione numerica con quella analitica nel piano xy al variare del passo Δt.

In particolare nella figura 1a si sono confrontate le soluzioni, al tempo `t = 100.0`, per un $\Delta t = 1.0$. Nella figura 1b il confronto lo si è fatto per $\Delta t = 0.01$. Il confronto con $\Delta t = 0.1$ mostra già una coincidenza tra le diverse soluzioni, esattamente come nella figura 1b.



(a) Soluzioni numeriche e soluzione esatta a confronto per un $\Delta t = 1.0$.



(b) Soluzioni numeriche e soluzione esatta a confronto per un $\Delta t = 0.01$.

Figura 1: Soluzioni numeriche e soluzione esatta a confronto per Δt differenti.

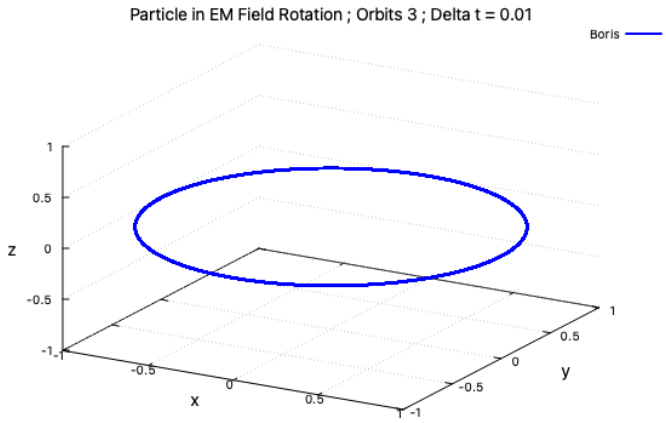


Figura 2: Test di convergenza dei due metodi numerici utilizzati a $t = 100.0$.

Dalle figura 1 è evidente che il metodo di Boris mostra una convergenza più accurata della soluzione, nonostante il minor ordine di convergenza, il quale lo possiamo verificare nella figura 2. Dalla figura del test di convergenza confermiamo quello che ci aspettiamo dalla teoria, ossia che il metodo di Runge-Kutta presenta una convergenza del quarto ordine, mentre l'algoritmo di Boris del secondo.

Infatti, con un $\Delta t = 1.0$ (figura 1a), seppur con qualche difficoltà di convergenza, l'algoritmo di Boris presenta una maggior efficienza rispetto il metodo di Runge-Kutta. Riducendo opportunamente il passo temporale a $\Delta t = 0.01$ per entrambi i metodi si ottiene una convergenza accurata della soluzione (figura 1b).

La miglior efficienza è dovuta al fatto che l'algoritmo di Boris è più adatto per problemi fisici in cui la forza dipende esplicitamente dalla velocità. C'è la discussione del metodo di Boris nella sezione 3.2.

Nella figura 3 si può vedere la traiettoria della particella in 3 dimensioni.

Tuttavia, come sappiamo dalla teoria, nonostante il quarto ordine del metodo di Runge-Kutta esso non preserva le caratteristiche geometriche del problema, al contrario dell'algoritmo di Boris. Infatti se si grafica, si veda la figura 4, l'energia cinetica della particella in funzione del tempo ai diversi timesteps:

$$\Delta t = \left(\frac{T}{60}\,,\,\frac{T}{80}\,,\,\frac{T}{100}\,,\,\frac{T}{200}\,,\,\frac{T}{1000}\right) = \big(1.66\,,\,1.25\,,\,1.0\,,\,0.5\,,\,0.1\big) \tag{17}$$

si vede che il metodo di Runge-Kutta non preservi l'energia per valori di Δt elevati, al contrario dell'algoritmo di Boris.

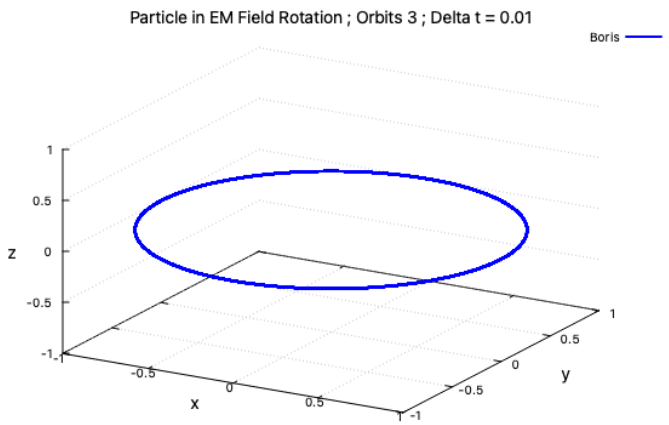


Figura 3: Traiettoria della particella in 3 dimensioni utilizzando l'algoritmo di Boris, a $t = 100.0$ con $\Delta t = 0.01$.

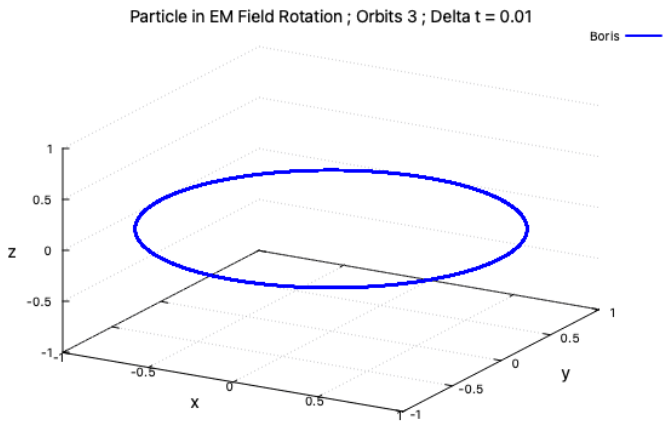


Figura 4: Confronto energie cinetiche della particella per $t = ...$

4.3 Campo elettrico e magnetico paralleli

La seconda configurazione studiata è quella in cui i campi elettrico e magnetico sono stati scelti:

$$\mathbf{E} = \big(0.0\,,\,0.0\,,\,0.6\big) \qquad ; \qquad \mathbf{B} = \big(0.0\,,\,0.0\,,\,1.0\big) \tag{18}$$

in cui si è scelto $|\mathbf{E}| < |\mathbb{B}|$ in modo da evitare i casi relativistici, che non sono interesse di questo progetto. L'equazione del moto in componenti (6) è:

$$\begin{aligned} \frac{\mathrm{d}v_x}{\mathrm{d}t} &= v_y B_z \\ \frac{\mathrm{d}v_y}{\mathrm{d}t} &= -v_x B_z \\ \frac{\mathrm{d}v_z}{\mathrm{d}t} &= E_z. \end{aligned} \tag{19}$$

Come condizioni iniziali su posizione e velocità delle particelle si sono scelte le stesse del caso precedente:

$$\mathbf{x} = \big(0.0\,,\,1.0\,,\,0.0\big) \qquad ; \qquad \mathbf{v} = \big(1.0\,,\,0.0\,,\,0.0\big).$$

La soluzione delle equazioni (19) è la stessa curva sul piano xy del caso precedente:

$$(x, y) = (\sin(\theta), \cos(\theta)) \tag{20}$$

in cui θ è sempre un parametro interpretabile come tempo di simulazione $\theta = t$, ma ora si ha anche la presenza di un moto accelerato, con $a_z = E_z$, lungo l'asse z. Dunque, la nostra particella percorrerà una traiettoria ad elica con passo variabile nel tempo. Si può vedere in figura 5 la traiettoria della particella nello spazio tridimensionale.

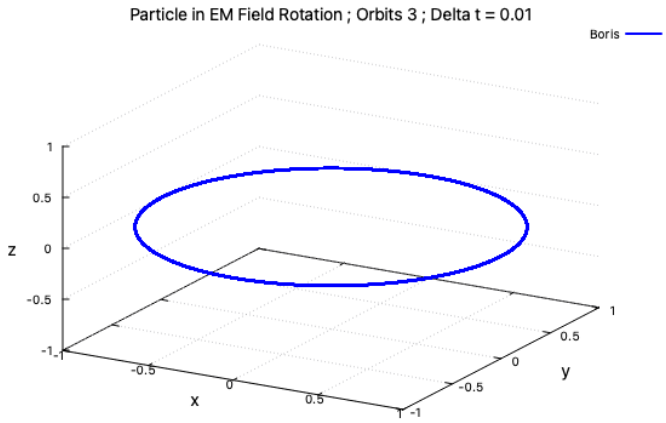


Figura 5: Traiettoria della particella in 3 dimensioni utilizzando l'algoritmo di Boris, a $t = 100.0$ con $\Delta t = 0.01$.

4.4 Campo elettrico e magnetico perpendicolari

Il terzo caso studiato è la configurazione perpendicolare di campo elettrico e magnetico:

$$\mathbf{E} = \big(0.0\,,\, 0.6\,,\, 0.0\big) \qquad ; \qquad \mathbf{B} = \big(0.0\,,\, 0.0\,,\, 1.0\big) \tag{21}$$

sempre con $|\mathbf{E}| < |\mathbb{B}|$. L'equazione del moto in componenti (6) è:

$$\begin{aligned} \frac{\mathrm{d}v_x}{\mathrm{d}t} &= v_y B_z \\ \frac{\mathrm{d}v_y}{\mathrm{d}t} &= E_y - v_x B_z \\ \frac{\mathrm{d}v_z}{\mathrm{d}t} &= 0. \end{aligned} \tag{22}$$

Le condizioni iniziali su posizione e velocità delle particelle sono scelte le stesse dei casi precedenti:

$$\mathbf{x} = \big(0.0\,,\, 1.0\,,\, 0.0\big) \qquad ; \qquad \mathbf{v} = \big(1.0\,,\, 0.0\,,\, 0.0\big).$$

Le soluzioni dell'equazione del moto (22) saranno sicuramente quelle di un moto solamente sul piano xy, visto che l'equazione legata a v_z ci dice che la velocità in quella direzione è costante al suo valore iniziale, ossia $v_z = 0.0$.

4.5 Configurazione ad X

5 Conclusioni

A Inserting Code

```
////////////////////////////////////////////////////////////////////////////////
//              Name    : Gabriele Cembalo                                    //
//              A.Y.    : 2025-2026                                           //
//              Project : Particle(s) in Electromagnetic Fields               //
//                                                                            //
//              Starting date : 24-12-2025                                    //
//              Ending date   : ..-..-2026                                    //
////////////////////////////////////////////////////////////////////////////////

#include <iostream>
#include <iomanip>
#include <cmath>
#include <ctime>
#include <fstream>
#include <cstdlib>

using namespace std;

////////////////////////////////////////////////////////////////////////////////
//                            Macro declaration                               //
////////////////////////////////////////////////////////////////////////////////
#define STAGE 4      // Choose the problem type (1 to 4)
                     // 1,2,3 : single particle
                     // 4     : multiple particles (N_Part)
#define NMAX_EQ 64   // Maximum number of elements (safety limit)
                     // to avoid variable-size arrays
#define N_Part 10000 // Number of particles in the multi-particle problem
#define X_L 1000.0   // Boundary half-width along X axis for STAGE 4
#define Y_L 1000.0   // Boundary half-width along Y axis for STAGE 4
#define TL 1000.0     // Simulation's final time

////////////////////////////////////////////////////////////////////////////////
//                          Functions declaration                             //
////////////////////////////////////////////////////////////////////////////////
void EB_Fields(double *, double *);
void InitialCondition(double *);
void RHSFunc(double, double *, double *, double *);
void RK4Step(double, double *, double *,
             void (*)(double, double *, double *, double *),
             double, int);
void BorisStep(double, double *, double *, double);

////////////////////////////////////////////////////////////////////////////////
//                       Main function implementation                         //
////////////////////////////////////////////////////////////////////////////////
int main(){

    ofstream fdata1 , fdata2 , fdata3;
    fdata1.open("../data/RK4.dat");   // Solutions file
    fdata2.open("../data/BORIS.dat"); // Solutions file
    fdata3.open("../data/ERR.dat");   // Error file

    fdata1 << setiosflags ( ios::scientific );
    fdata1 << setprecision ( 10 );
    fdata2 << setiosflags ( ios::scientific );
    fdata2 << setprecision ( 10 );
    fdata3 << setiosflags ( ios::scientific );
    fdata3 << setprecision ( 10 );

    cout << setiosflags ( ios::scientific );
    cout << setprecision ( 7 );

    int neq = 6; // Number of equations
    int i , j;   // Loop index
    int k = 0;   // Control variable to check if in multiparticle problem they
                 // go outside the EB field region.

```

```
    int n_time = 3;  // Number of different time step

    double t = 0.0;                       // Time variable
    double dt = 0.01;                     // Time step
    double dt_vec[] = {1.0, 0.1, 0.01};   // Vector to change the time step
    double errRK4 = 0.0 , errBor = 0.0;   // Error variables

    // Uncomment the following vector to use it in the Kinetic energy plot:
    //double dt_vec[] = {1.66, 1.25, 1.0, 0.5, 0.1};

    double YRK4[neq] , YBor[neq]; // Solutions vector (for each method)
                                  // (x, y, z)    = (Y[0], Y[1], Y[2])
                                  // (vx, vy, vz) = (Y[3], Y[4], Y[5])

    double EB[neq]; // Electromagnetic field
                    // (Ex, Ey, Ez) = (EB[0], EB[1], EB[2])
                    // (Bx, By, Bz) = (EB[3], EB[4], EB[5])

    #if STAGE == 1 || STAGE == 2 || STAGE == 3  // Single particle problem

    // Solve the equation with each method with different time step
    // The ElectroMagnetic field will be set inside each method
    for( i = 0 ; i < n_time ; i++ ){

        dt = dt_vec[i]; // Set the time step

        // Set the initial condition
        InitialCondition(YRK4);
        InitialCondition(YBor);

        t = 0.0;

        // Print the initial condition (t, x, y, z, 0.5 v^2)
        fdata1 << t << "  " << YRK4[0] << "  " << YRK4[1] << "  " << YRK4[2]
               << "  "
               << 0.5*(YRK4[3]*YRK4[3] + YRK4[4]*YRK4[4] + YRK4[5]*YRK4[5])
               << endl;
        fdata2 << t << "  " << YBor[0] << "  " << YBor[1] << "  " << YBor[2]
               << "  "
               << 0.5*(YBor[3]*YBor[3] + YBor[4]*YBor[4] + YBor[5]*YBor[5])
               << endl;

        // Solve the equation
        for( j = 0 ; t < TL ; j++ ){

            RK4Step(t, YRK4, EB, RHSFunc, dt, neq);
            BorisStep(t, YBor, EB, dt);

            t += dt; // increasing time

            // Print in the data file (t, x, y, z, 0.5 v^2)
            fdata1 << t << "  " << YRK4[0] << "  " << YRK4[1] << "  " << YRK4[2]
                   << "  "
                   << 0.5*(YRK4[3]*YRK4[3] + YRK4[4]*YRK4[4] + YRK4[5]*YRK4[5])
                   << endl;
            fdata2 << t << "  " << YBor[0] << "  " << YBor[1] << "  " << YBor[2]
                   << "  "
                   << 0.5*(YBor[3]*YBor[3] + YBor[4]*YBor[4] + YBor[5]*YBor[5])
                   << endl;

        }

        fdata1 << endl << endl; // Skip 2 line in the data file
        fdata2 << endl << endl;

        // Compute the error as the difference between numerical
        // and analytical solutions at t.
        errRK4 = sqrt( (YRK4[0] - sin(t))*(YRK4[0] - sin(t)) +
                       (YRK4[1] - cos(t))*(YRK4[1] - cos(t)) );
```

```
        errBor = sqrt( (YBor[0] - sin(t))*(YBor[0] - sin(t)) +
                       (YBor[1] - cos(t))*(YBor[1] - cos(t)) );

        // Print the error in data file
        fdata3 << dt << "  " << errRK4 << "  " << errBor << endl;

    }

    #elif STAGE == 4  // Multiparticle problem

    // We need to generate some random quantities (used in InitialCondition)
    srand48(time(NULL));

    // Print the initial condition
    for( i = 0 ; i < N_Part ; i++ ){

        // Set the initial condition
        InitialCondition(YRK4);
        InitialCondition(YBor);

        // Print in the data file (t, x, y, z, 0.5 v^2)
        fdata1 << t << "  " << YRK4[0] << "  " << YRK4[1] << "  " << YRK4[2]
               << "  "
               << 0.5*(YRK4[3]*YRK4[3] + YRK4[4]*YRK4[4] + YRK4[5]*YRK4[5])
               << endl;
        fdata2 << t << "  " << YBor[0] << "  " << YBor[1] << "  " << YBor[2]
               << "  "
               << 0.5*(YBor[3]*YBor[3] + YBor[4]*YBor[4] + YBor[5]*YBor[5])
               << endl;

    }

    fdata1 << endl << endl; // Skip 2 line in the data file
    fdata2 << endl << endl;

    // Solve the equation with each method with different time step
    // The ElettroMagnetic field will be set inside each method
    for( i = 0 ; i < N_Part ; i++ ){

        // Set the initial condition
        InitialCondition(YRK4);

        t = 0.0; // Setting the starting time

        // Solve the equation
        for( j = 0 ; t < TL ; j++ ){

            k = 0;

            // Use 'k' as a flag to check if the particle leaves the
            // computational domain. If so, we break the loop.
            if( YRK4[0] < -X_L || YRK4[0] > X_L
                || YRK4[1] < -Y_L || YRK4[1] > Y_L ){

                    k = 1;
                    break;

            }

            RK4Step(t, YRK4, EB, RHSFunc, dt, neq);

            t += dt; // increasing time

        }

        if( k == 0 ){

            // Print in the data file (t, x, y, z, 0.5 v^2)
            fdata1 << t << "  " << YRK4[0] << "  " << YRK4[1] << "  " << YRK4[2]
```

```
                    << "   "
                    << 0.5*(YRK4[3]*YRK4[3] + YRK4[4]*YRK4[4] + YRK4[5]*YRK4[5])
                    << endl;

        }

    }

    for( i = 0 ; i < N_Part ; i++ ){

        // Set the initial condition
        InitialCondition(YBor);

        t = 0.0; // Setting the starting time

        // Solve the equation
        for( j = 0 ; t < TL ; j++ ){

            k = 0;

            // We'll use the integer k to check if our solution are going
            // outside the integral region. If so, we break the loop.
            if( YBor[0] < -X_L || YBor[0] > X_L
                || YBor[1] < -Y_L || YBor[1] > Y_L ){

                    k = 1;
                    break;

            }

            BorisStep(t, YBor, EB, dt);

            t += dt; // Increasing time

        }

        if( k == 0 ){

            // Print in the data file (t, x, y, z, 0.5 v^2)
            fdata2 << t << "   " << YBor[0] << "   " << YBor[1] << "   " << YBor[2]
                    << "   "
                    << 0.5*(YBor[3]*YBor[3] + YBor[4]*YBor[4] + YBor[5]*YBor[5])
                    << endl;

        }

    }

    #endif

    fdata1.close();
    fdata2.close();
    fdata3.close();

    return 0;

}

////////////////////////////////////////////////////////////////////////////////
//                          Functions implementation                          //
////////////////////////////////////////////////////////////////////////////////


////////////////////////////////////////////////////////////////////////////////
// Electric and magnetic fields.                                              //
// Input: pointer to the solution vector,                                     //
//        pointer to the electric and magnetic field vector.                 //
////////////////////////////////////////////////////////////////////////////////
void EB_Fields(double *Y, double *EB){
```

```

    double l = 0.001; // Value of 1/L, where L = X_L = Y_L

    #if STAGE == 1 // Single particle problem
                   // Simple gyration
        EB[0] = 0.0 ; EB[1] = 0.0 ; EB[2] = 0.0;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 2 // Single particle problem
                     // ExB drift in parallel configuration ( |E| < |B| )
        EB[0] = 0.0 ; EB[1] = 0.0 ; EB[2] = 0.6;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 3 // Single particle problem
                     // ExB drift in perpendicular configuration ( |E| < |B| )
        EB[0] = 0.0 ; EB[1] = 0.6 ; EB[2] = 0.0;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 4 // Multiple particles problem
                     // X point configuration
        EB[0] = 0.0    ; EB[1] = 0.0    ; EB[2] = 0.5;
        EB[3] = Y[1]*l ; EB[4] = Y[0]*l ; EB[5] = 0.0;

    #endif

}


////////////////////////////////////////////////////////////////////////////
// Initial condition.                                                     //
// Input: pointer to the solution vector.                                 //
////////////////////////////////////////////////////////////////////////////
void InitialCondition(double *Y){

    #if STAGE == 1 || STAGE == 2 || STAGE == 3 // Single particle problem
        Y[0] = 0.0 ; Y[1] = 1.0 ; Y[2] = 0.0;  // Initial position
        Y[3] = 1.0 ; Y[4] = 0.0 ; Y[5] = 0.0;  // Initial velocity

    #elif STAGE == 4 // Multiple particle problem
        // Initialize random positions and velocities with fixed magnitude.
        double x, y, v, vx, vy, theta;
        v = 0.1; // velocity magnitude

        // Generating random (x, y) coordinates in the range [-1000, 1000]^2
        x = (drand48()*2.0*X_L) - X_L;
        y = (drand48()*2.0*Y_L) - Y_L;

        theta = drand48()*2*M_PI; // Random angle in [0,2*pi]

        // Random velocity with magnitude v and angle theta
        vx = v*cos(theta);
        vy = v*sin(theta);

        // Filling the state vector
        Y[0] = x  ; Y[1] = y  ; Y[2] = 0.0; // Initial position
        Y[3] = vx ; Y[4] = vy ; Y[5] = 0.0; // Initial velocity

    #endif

}


////////////////////////////////////////////////////////////////////////////
// Right-Hand-Side function of the equation.                              //
// Input: integration variable,                                           //
//        pointer to the solution vector,                                 //
//        pointer to the electric and magnetic field vector,              //
//        pointer to the RHS vector.                                      //
////////////////////////////////////////////////////////////////////////////
```

```
void RHSFunc(double t, double *Y, double *EB, double *R){

    // Using natural units: c = 1

    EB_Fields(Y, EB); // Fill the EB vector

    // Assigning variables
    double x = Y[0]   , y = Y[1]   , z = Y[2];
    double vx = Y[3]  , vy = Y[4]  , vz = Y[5];
    double Ex = EB[0] , Ey = EB[1] , Ez = EB[2];
    double Bx = EB[3] , By = EB[4] , Bz = EB[5];

    R[0] = vx;
    R[1] = vy;
    R[2] = vz;
    R[3] = Ex + vy*Bz - vz*By;
    R[4] = Ey + vz*Bx - vx*Bz;
    R[5] = Ez + vx*By - vy*Bx;

}


////////////////////////////////////////////////////////////////////////////
// Fourth-order Runge-Kutta method.                                       //
// Input: integration variable,                                           //
//        pointer to the solution vector,                                 //
//        pointer to the electric and magnetic field vector,              //
//        pointer to the Right-Hand-Side-Function,                        //
//        step size,                                                      //
//        order of the ODE system.                                        //
////////////////////////////////////////////////////////////////////////////
void RK4Step(double t, double *Y, double *EB,
             void (*RHSFunc)(double t, double *Y, double *EB, double *R),
             double h, int neq){

    int i; // Loop index
    // Vectors for intermediate steps
    double Y1[NMAX_EQ], k1[NMAX_EQ], k2[NMAX_EQ], k3[NMAX_EQ], k4[NMAX_EQ];

    RHSFunc(t, Y, EB, k1); // Computation of k1 using RHS at t_n and Y_n

    // Loop to compute Y_n + k1*h/2
    for( i = 0 ; i < neq ; i++ ){

        Y1[i] = Y[i] + 0.5*h*k1[i];

    }

    RHSFunc(t+0.5*h, Y1, EB, k2); // Computation of k2 using RHS at t_n + h/2
                                  // and Y_n + k1*h/2

    // Loop to compute Y_{n+1} = Y_n + k2*h/2
    for( i = 0 ; i < neq ; i++ ){

        Y1[i] = Y[i] + h*k2[i]*0.5;

    }

    RHSFunc(t+0.5*h, Y1, EB, k3); // Computation of k3 using RHS at t_n + h/2
                                  // and Y_n + k2*h/2

    // Loop to compute Y_{n+1} = Y_n + k3*h
    for(i = 0 ; i < neq ; i++){

        Y1[i] = Y[i] + h*k3[i];

    }

    RHSFunc(t+h, Y1, EB, k4); // Computation of k4 using RHS at t_n + h
```

```
                                // and Y_n + k3*h

    // Loop to compute Y_{n+1} = Y_n + h/6 * ( k1 + 2*k2 + 2*k3 + k4 )
    for(i = 0 ; i < neq ; i++){

        Y[i] += h * ( k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i] )/6.0;

    }

}


////////////////////////////////////////////////////////////////////////////////
// Boris Method Step (second-order, symplectic)                               //
// Input: integration variable,                                               //
//        pointer to the solution vector,                                     //
//        pointer to the electric and magnetic field vector,                 //
//        step size.                                                          //
////////////////////////////////////////////////////////////////////////////////
void BorisStep(double t, double *Y, double *EB, double h){

    // Reminder: (x, y, z)    = (Y[0], Y[1], Y[2])
    //           (vx, vy, vz) = (Y[3], Y[4], Y[5])
    //           (Ex, Ey, Ez) = (EB[0], EB[1], EB[2])
    //           (Bx, By, Bz) = (EB[3], EB[4], EB[5])

    // We'll compute the following step: 1. x_{n+1/2} = x_n + v_n * h/2
    //                                   2. s and t
    //                                   3. v_minus = v_n + E * h/2
    //                                   4. v_prime and v_plus
    //                                   5. v_{n+1} = v_plus + E * h/2
    //                                   6. x_{n+1} = x_{n+1/2} + v_{n+1} * h/2

    int i; // Loop index
    // Vectors for intermediate steps
    double v_minus[3], v_plus[3], t_vec[3], s_vec[3], v_prime[3];
    // Where: t_vec = B * h * 0.5
    //        s_vec = 2t / ( 1 + |t|^2 )
    //        v^prime = v_- + v_- cross t
    double t_mag; // This is the magnitude ^2 of t_vec

    EB_Fields(Y, EB); // Fill the EB vector

    // 1. Compute the position half-step: x_{n+1/2} = x_n + v_n * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i] = Y[i] + Y[i+3] * 0.5 * h;

    }

    // 2. Compute the intermediate vectors for the magnetic rotation
    t_mag = 0;
    for( i = 0 ; i < 3 ; i++ ){

        t_vec[i] = EB[i+3] * 0.5 * h;
        t_mag += t_vec[i] * t_vec[i];

    }
    for( i = 0 ; i < 3 ; i++ ){

        s_vec[i] = 2.0 * t_vec[i] / (1.0 + t_mag);

    }

    // 3. Compute the velocity v_- step (electric field):
    //    v_minus = v_n + E * h/2
    for( i = 0 ; i < 3 ; i++ ){

        v_minus[i] = Y[i+3] + EB[i] * 0.5 * h;
```

```
    }

    // 4. Magnetic rotation (Boris Trick)
    //     v_prime = v_minus + v_minus cross t
    v_prime[0] = v_minus[0] + (v_minus[1] * t_vec[2] - v_minus[2] * t_vec[1]);
    v_prime[1] = v_minus[1] + (v_minus[2] * t_vec[0] - v_minus[0] * t_vec[2]);
    v_prime[2] = v_minus[2] + (v_minus[0] * t_vec[1] - v_minus[1] * t_vec[0]);

    // Compute: v_plus = v_minus + v_prime x s
    v_plus[0] = v_minus[0] + (v_prime[1] * s_vec[2] - v_prime[2] * s_vec[1]);
    v_plus[1] = v_minus[1] + (v_prime[2] * s_vec[0] - v_prime[0] * s_vec[2]);
    v_plus[2] = v_minus[2] + (v_prime[0] * s_vec[1] - v_prime[1] * s_vec[0]);

    // 5. Compute the second velocity step (electric field):
    //     v_{n+1} = v_plus + E * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i+3] = v_plus[i] + EB[i] * 0.5 * h;

    }

    // 6. Compute the position step: x_{n+1} = x_{n+1/2} + v_{n+1} * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i] = Y[i] + Y[i+3] * 0.5 * h;

    }

}

////////////////////////////////////////////////////////////////////////////////
//                              Plot parameter                                //
////////////////////////////////////////////////////////////////////////////////
// I list the parameter I've used to plot the data (for each configuration).  //
// Rotation (STAGE 1):                                                        //
//           Err vs dt          TL = 100                                      //
//           Kin en vs t        TL = 100                                      //
//                              different dt vector (see it in the code)      //
//           Orbit (x,y)        TL = 100                                      //
//           3D Orbit (x,y)     TL = 100                                      //
// EM Parallel (STAGE 2):                                                     //
//           3D Orbit (x,y)     TL = 100                                      //
// EM Perpendicular (STAGE 3):                                                //
//           Orbit (x,y)        TL = 100                                      //
//           3D Orbit (x,y)     TL = 100                                      //
// X point (STAGE 4):                                                         //
//           Kin en vs (x,y)    TL      = 0 , 300 , 1000                      //
//                              N_PART  = 3000                                //
//           Istogram Kin en    TL      = 1000                                //
//                              Scaling = 0.1                                 //
//                              N_PART = 3000                                 //
//           Power law          TL     = 1000                                 //
//                              N_PART = 10000                                //
////////////////////////////////////////////////////////////////////////////////
```

Riferimenti bibliografici

[1] Particle In Cell Consulting. *The Boris Method: $\boldsymbol{v} \times \boldsymbol{B}$ Rotation*. https://www.particleincell.com/2011/vxb-rotation/. 2011.

[2] Hong Qin et al. «Why is Boris algorithm so good?» In: *Physics of Plasmas* 20.8 (2013), p. 084503. DOI: 10.1063/1.4818428.